```
/**
 * Payload Decoder for Datacake
 * Based on Milesight EM500-SMT payload format
 * Copyright 2023 Milesight IoT
 *
 * @product EM500-SMT
 * Sample payload: 01756403671901046873057FF000
 */
function Decoder(bytes, port) {
    return milesight(bytes);
}

function milesight(bytes) {
    var decoded = {};

    for (var i = 0; i < bytes.length; ) {

        // Guard: need at least 2 bytes for channel_id + channel_type
        if (i + 1 >= bytes.length) break;

        var channel_id   = bytes[i++];
        var channel_type = bytes[i++];

        // BATTERY
        if (channel_id === 0x01 && channel_type === 0x75) {
            if (i + 1 > bytes.length) break;
            decoded.battery = bytes[i];
            i += 1;
        }
        // TEMPERATURE
        else if (channel_id === 0x03 && channel_type === 0x67) {
            if (i + 2 > bytes.length) break;
            decoded.temperature = readInt16LE(bytes.slice(i, i + 2)) / 10;
            i += 2;
        }
        // MOISTURE (old resolution 0.5)
        else if (channel_id === 0x04 && channel_type === 0x68) {
            if (i + 1 > bytes.length) break;
            decoded.moisture = bytes[i] / 2;
            i += 1;
        }
        // MOISTURE (new resolution 0.01)
        else if (channel_id === 0x04 && channel_type === 0xca) {
            if (i + 2 > bytes.length) break;
            decoded.moisture = readInt16LE(bytes.slice(i, i + 2)) / 100;
            i += 2;
        }
        // EC
        else if (channel_id === 0x05 && channel_type === 0x7f) {
            if (i + 2 > bytes.length) break;
            decoded.ec = readUInt16LE(bytes.slice(i, i + 2));
            i += 2;
        }
        // HISTORY DATA
        else if (channel_id === 0x20 && channel_type === 0xce) {
            if (i + 10 > bytes.length) break;

            // ← was overwriting top-level decoded fields; now stored as an array of objects
            if (!decoded.history) decoded.history = [];
            decoded.history.push({
```

```
            timestamp:   readUInt32LE(bytes.slice(i,     i + 4)),
            ec:          readInt16LE (bytes.slice(i + 4, i + 6)),
            temperature: readInt16LE (bytes.slice(i + 6, i + 8)) / 10,
            moisture:    readInt16LE (bytes.slice(i + 8, i + 10)) / 100
        });
        i += 10;
    }
    // Unknown channel — skip gracefully instead of discarding rest of payload
    else {
        i++;
    }
}

// Flatten history array into indexed scalar fields — Datacake cannot store arrays
if (decoded.history && decoded.history.length > 0) {
    decoded.history.forEach(function(entry, idx) {
        var prefix = "history_" + (idx + 1) + "_";
        Object.keys(entry).forEach(function(key) {
            decoded[prefix + key] = entry[key];
        });
    });
    delete decoded.history;
}

// LoRa radio metadata
try {
    var gw = (normalizedPayload.gateways && normalizedPayload.gateways[0]) || {};
    decoded.lora_rssi     = gw.rssi || 0;
    decoded.lora_snr      = gw.snr  || 0;
    decoded.lora_datarate = normalizedPayload.data_rate || "not retrievable";
} catch (e) {
    decoded.lora_rssi     = 0;
    decoded.lora_snr      = 0;
    decoded.lora_datarate = "not retrievable";
}

// Datacake expects an array of { field, value } objects with uppercase field names
return Object.keys(decoded).map(function(key) {
    return { field: key.toUpperCase(), value: decoded[key] };
});
}

/* ****************************************
 * bytes to number helpers
 ****************************************/
function readUInt16LE(bytes) {
    var value = (bytes[1] << 8) + bytes[0];
    return value & 0xffff;
}
function readInt16LE(bytes) {
    var ref = readUInt16LE(bytes);
    return ref > 0x7fff ? ref - 0x10000 : ref;
}
function readUInt32LE(bytes) {
    var value = (bytes[3] << 24) + (bytes[2] << 16) + (bytes[1] << 8) + bytes[0];
    return (value & 0xffffffff) >>> 0; // ← added >>> 0 to ensure unsigned result
}
```